

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T08:23:51.1012152Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-082351205.prof
Profil mémoire	memory-20260924-082351205.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	133.81 ms	133805162	0	0

Détail hyperfine

Métrique	Valeur
Moyenne	2.6701 s
Écart-type	n/a
Médiane	2.6701 s
Temps utilisateur	1.5431 s
Temps système	0.9019 s
Min	2.6701 s
Max	2.6701 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	1.21s
Mémoire allouée (pprof)	20399.75kB
Mémoire par opération	0 B/op

Garbage collector Go

Cycles GC	58
Temps GC cumulé	58.080 ms

Plus longue pause	2.700 ms
Temps CPU GC cumulé	314.670 ms
Objectif mémoire maximal	97.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **1.21s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).shouldInfect	0.95s	78.51%	1.17s	96.69%
jeu-de-la-vie/game.(*Board).index (inline)	0.15s	12.40%	0.15s	12.40%
jeu-de-la-vie/game.squaredDistance (inline)	0.05s	4.13%	0.05s	4.13%
math/rand.(*Rand).Int63 (inline)	0.02s	1.65%	0.02s	1.65%
runtime.cgocall	0.02s	1.65%	0.02s	1.65%
jeu-de-la-vie/game.(*Board).Step	0.01s	0.83%	1.18s	97.52%
runtime.schedule	0.01s	0.83%	0.01s	0.83%
internal/syscall/windows/registry.Key.GetMUISt	0	0.00%	0.02s	1.65%

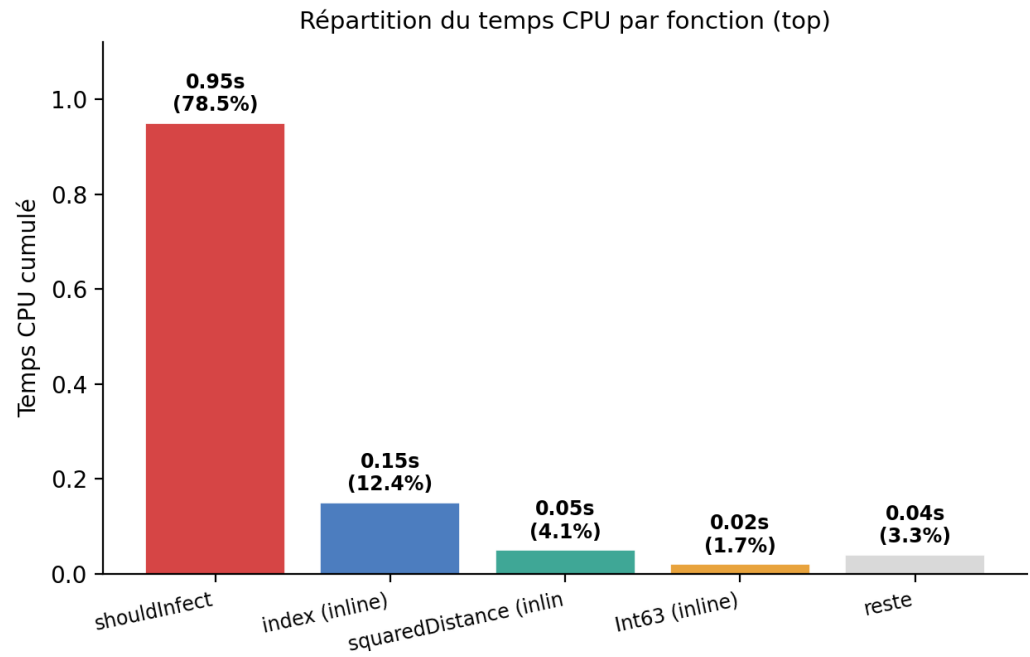


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

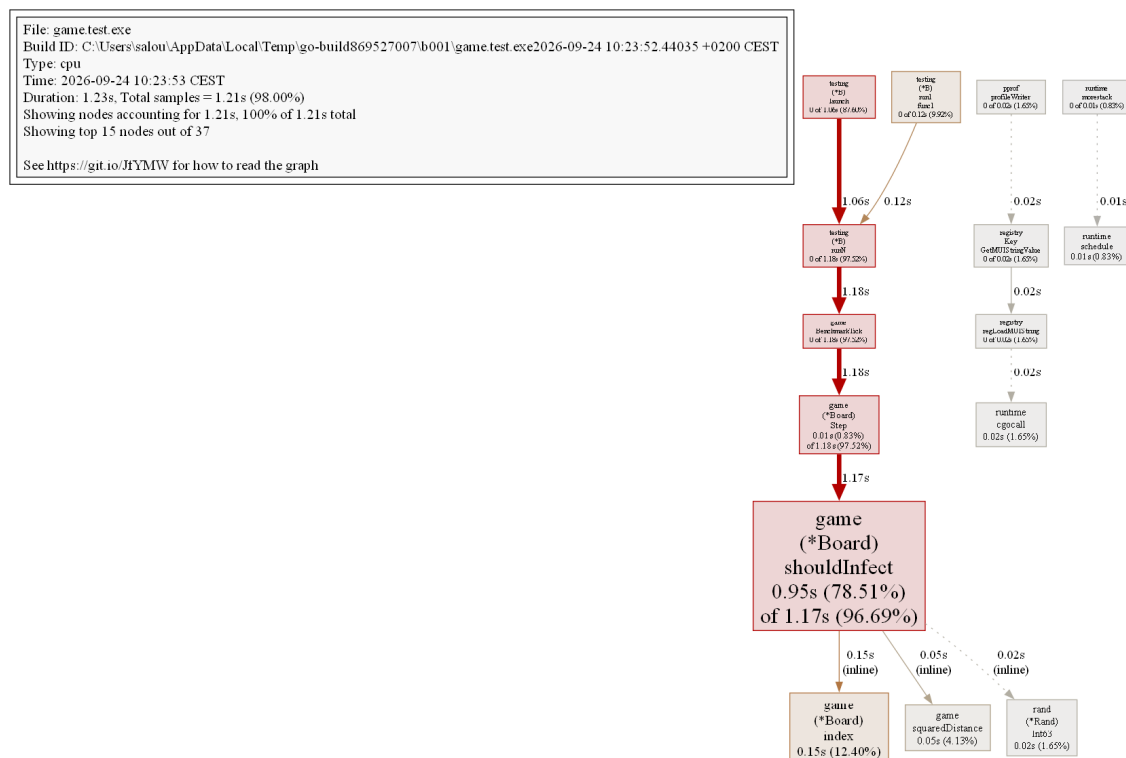


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).shouldInfect**, responsable de **78.5%** du temps CPU total (flat), pour **96.7%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine à mesure que les données grandissent (schéma, non mesuré)

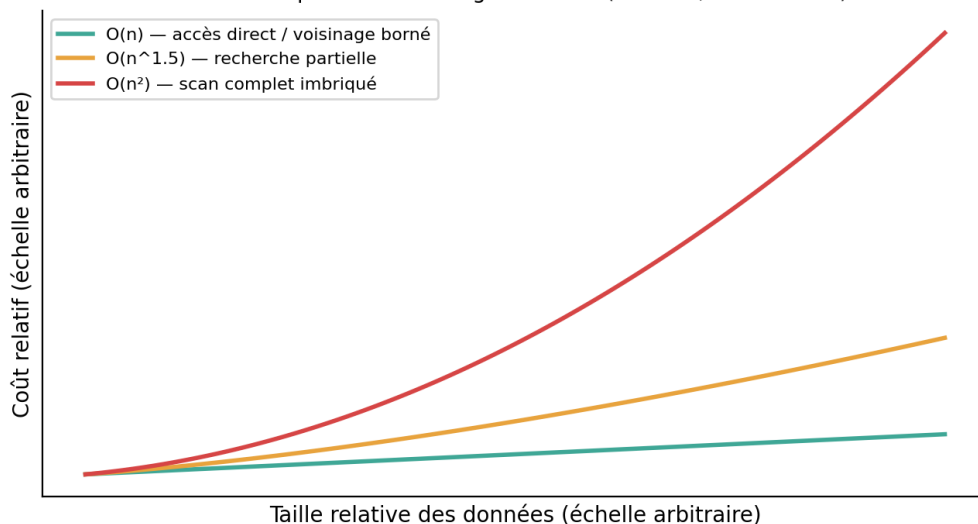


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **20399.75kB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.NewBoard (inline)	8999.20kB	44.11%	8999.20kB	44.11%
jeu-de-la-vie/game.RandomBoard	5655.11kB	27.72%	14654.31kB	71.84%
runtime.mallocgc	1539kB	7.54%	1539kB	7.54%
runtime/pprof.StartCPUProfile	1184.27kB	5.81%	1184.27kB	5.81%
compress/flate.NewWriter (inline)	902.59kB	4.42%	1485.59kB	7.28%
compress/flate.newDeflateFast (inline)	583.01kB	2.86%	583.01kB	2.86%
hash/crc32.simpleMakeTable (inline)	512.50kB	2.51%	512.50kB	2.51%
syscall.ByteSliceFromString	512.05kB	2.51%	512.05kB	2.51%

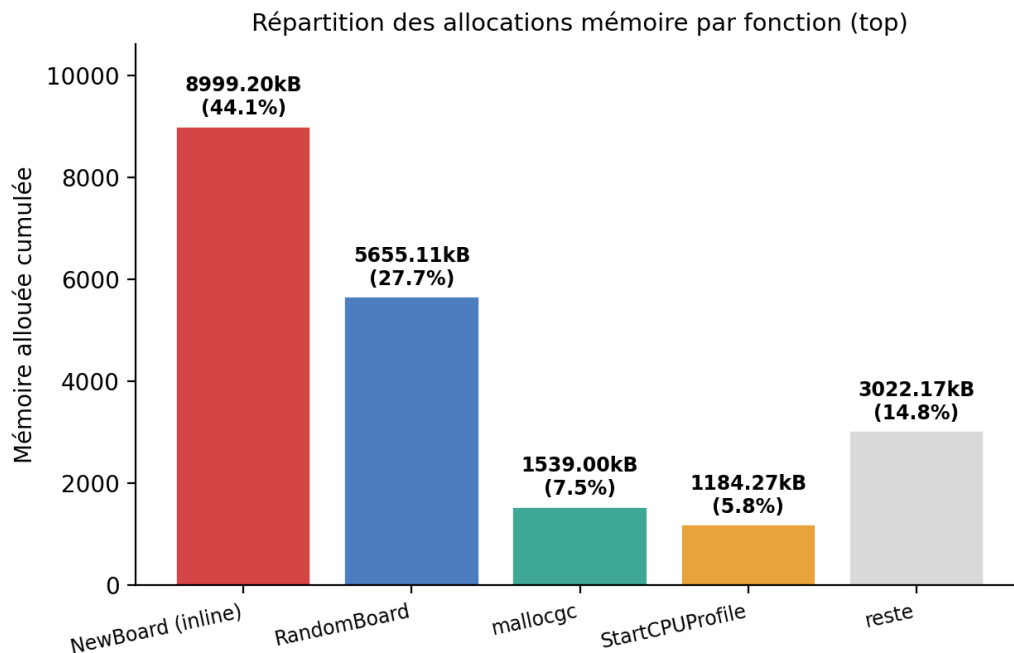


Figure — Répartition des allocations mémoire par fonction.

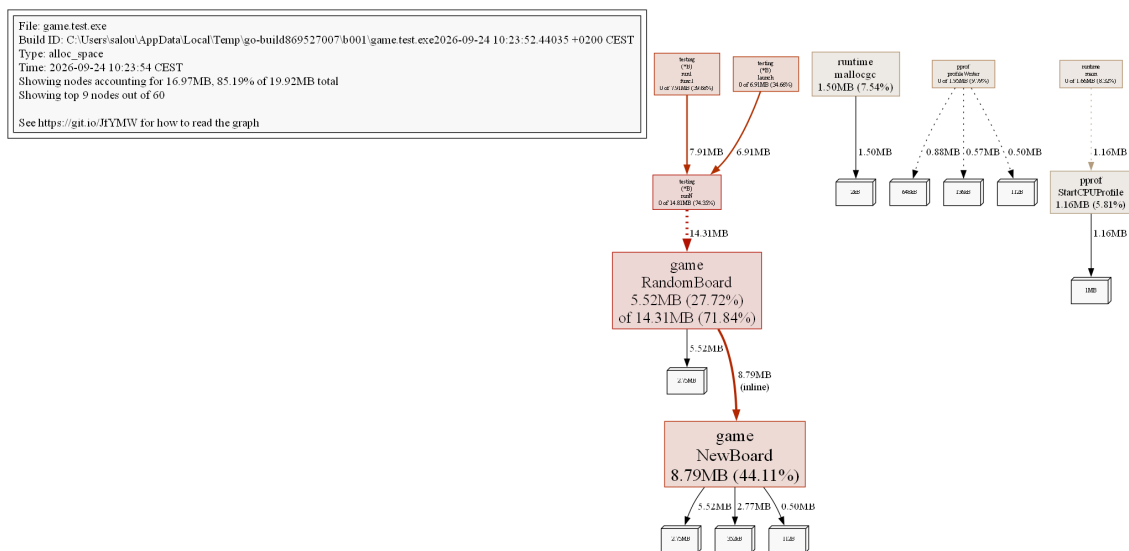


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 10:24:07 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.